Here is how i installed and configure DWVA

Setting up **DVWA** (Damn Vulnerable Web Application) is an essential rite of passage for anyone getting into web security. Because it is intentionally full of holes, the installation requires a few specific "tweaks" to ensure everything works as intended.

I configured the vulnerable dvwa on a kali-linux virtual machine to ensure *testing and configuration are done in a secure and safe environment.*

Here is a step-by-step documentation

Prerequisites

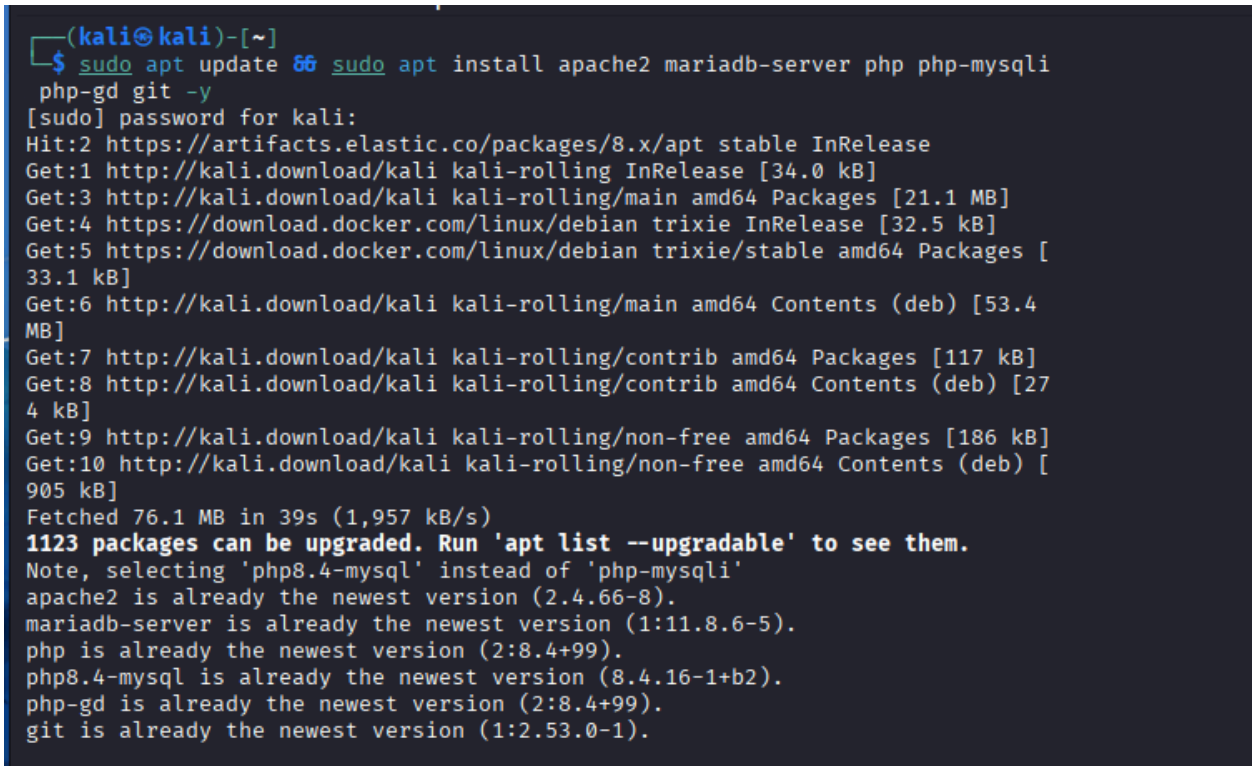
Before jumping to installation they are required packages to ensure they are installed and configured.

- **Web Server:** Apache2
- **Database:** MariaDB (or MySQL)
- **Language:** PHP (plus the php-gd extension for image processing)

Firstly:

Updating our kali with this command and download the package

Sudo apt update && sudo apt update && sudo apt install apache2 mariadb-server php php-mysqli php-gd git -y

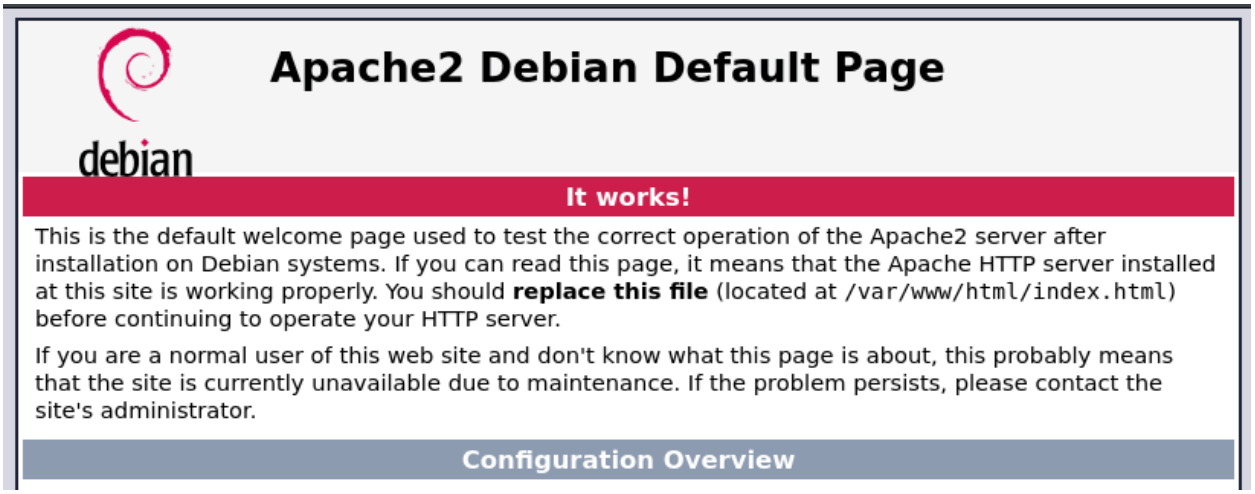


Confirming apache2 is working as needed

Sudo systemctl start apache2

Go to localhost or 127.0.0.1 on my browser

If the below interface appears it works!



Step 1: Download and Deploy DVWA

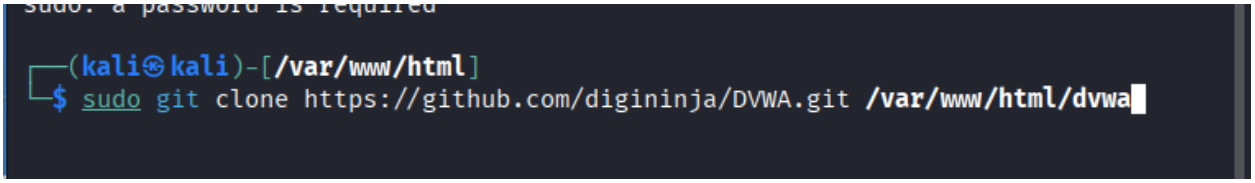
We will clone the source code directly from GitHub into the web server's root directory. before that we need to Navigate to the web server folder.

Navigate to the web directory:

cd /var/www/html

Clone the repository:

sudo git clone https://github.com/digininja/DVWA.git /var/www/html/dvwa



Navigate to the folder

```
┌──(kali㉿kali)-[/var/www/html/dvwa]
└─$ ls
about.php       dvwa              phpinfo.php   README.ko.md  SECURITY.md
CHANGELOG.md    external          php.ini       README.md     security.php
compose.yml     favicon.ico       README.ar.md  README.pl.md  security.txt
config          hackable          README.es.md  README.pt.md  setup.php
COPYING.txt     index.php         README.fa.md  README.tr.md  tests
database        instructions.php  README.fr.md  README.vi.md  vulnerabilities
Dockerfile      login.php         README.id.md  README.zh.md
docs            logout.php        README.it.md  robots.txt
```

Set Permissions

We need to set full permission (read, write, execute) to the owner while limiting access to others and groups. This follow the principle of least priviledge

sudo chown -R www-data:www-data /var/www/html/dvwa

sudo chmod -R 755 /var/www/html/dvwa

```
┌──(kali㉿kali)-[/var/www/html/dvwa]
└─$ sudo chown -R www-data:www-data /var/www/html/dvwa
sudo chmod -R 755 /var/www/html/dvwa
```

Step 2: Configure the Database

DVWA needs a database to store users and vulnerability data.

Access MariaDB:

sudo mysql -u root

Run these commands inside the MySQL prompt to create a dedicated user:

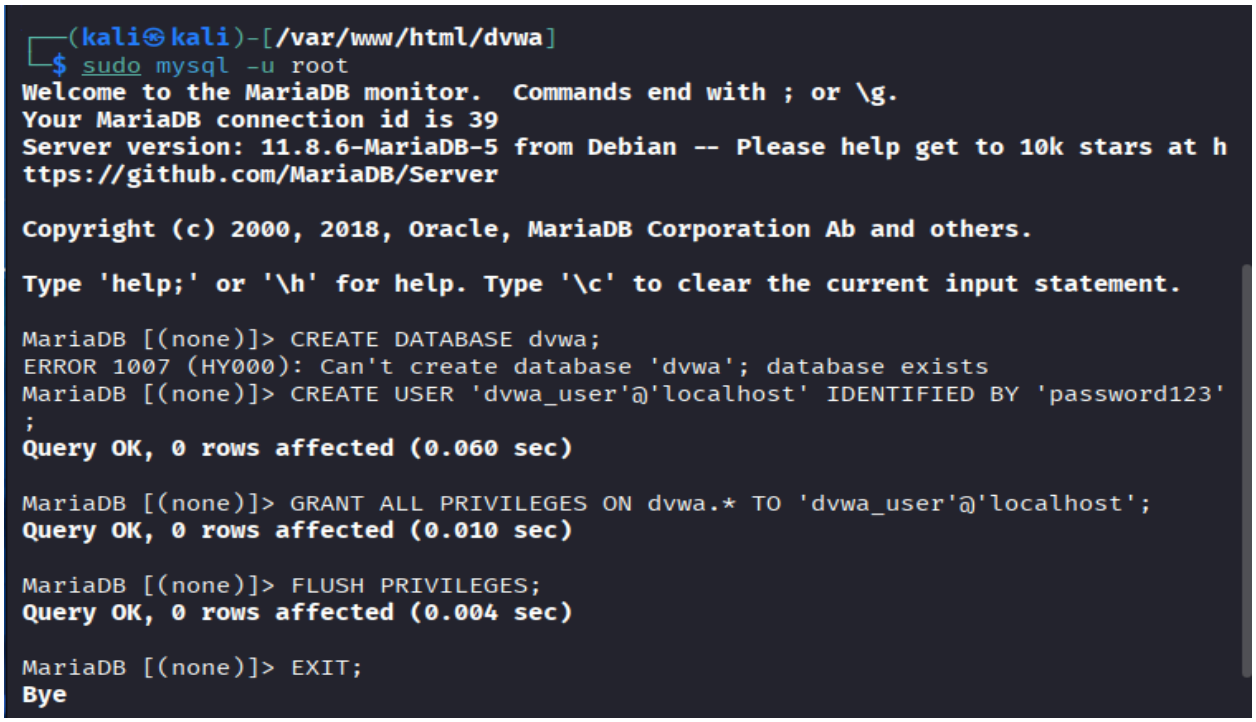
CREATE DATABASE dvwa;

CREATE USER 'dvwa_user'@'localhost' IDENTIFIED BY 'password123';

GRANT ALL PRIVILEGES ON dvwa. TO 'dvwa_user'@'localhost';*

FLUSH PRIVILEGES;

EXIT;



Step 3: Configure DVWA Settings

DVWA comes with a "dist" (distribution) config file that we must rename and edit.

Using the following command

cd /var/www/html/DVWA/config

sudo cp config.inc.php.dist config.inc.php

```
┌──(kali㉿kali)-[/var/www/html/dvwa/config]
└─$ sudo cp config.inc.php.dist config.inc.php

┌──(kali㉿kali)-[/var/www/html/dvwa/config]
└─$ ls
config.inc.php  config.inc.php.bak  config.inc.php.dist
```

Editing the file:

sudo nano config.inc.php

Update the Database Credentials to match what i created in Step 2 and save file:

$_DVWA['db_user'] = 'dvwa_user';

$_DVWA['db_password'] = 'password123';

```
  GNU nano 8.7                          config.inc.php *
<?php

# If you are having problems connecting to the MySQL database and all of the var>
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a pro>
#   Thanks to @digininja for the fix.

# Database management system to use
$DBMS = getenv('DBMS') ?: 'MySQL';
#$DBMS = 'PGSQL'; // Currently disabled

# Database variables
#   WARNING: The database specified under db_database WILL BE ENTIRELY DELETED d>
#   Please use a database dedicated to DVWA.
#
# If you are using MariaDB then you cannot use root, you must use create a dedic>
#   See README.md for more information on this.
$_DVWA = array();
$_DVWA[ 'db_server' ]   = getenv('DB_SERVER') ?: '127.0.0.1';
$_DVWA[ 'db_database' ] = getenv('DB_DATABASE') ?: 'dvwa';
$_DVWA[ 'db_user' ]     = getenv('DB_USER') ?: 'dvwa_user';
$_DVWA[ 'db_password' ] = getenv('DB_PASSWORD') ?: 'p@ssw0rd123';
$_DVWA[ 'db_port']      = getenv('DB_PORT') ?: '3306';

# ReCAPTCHA settings
```

Step 4: configuring php and apache2 (Crucial Step)

Enabling specific PHP settings for For certain vulnerabilities (like File Inclusion)

used the bellow command to look for the config file:

php -i | grep "Loaded Configuration File"



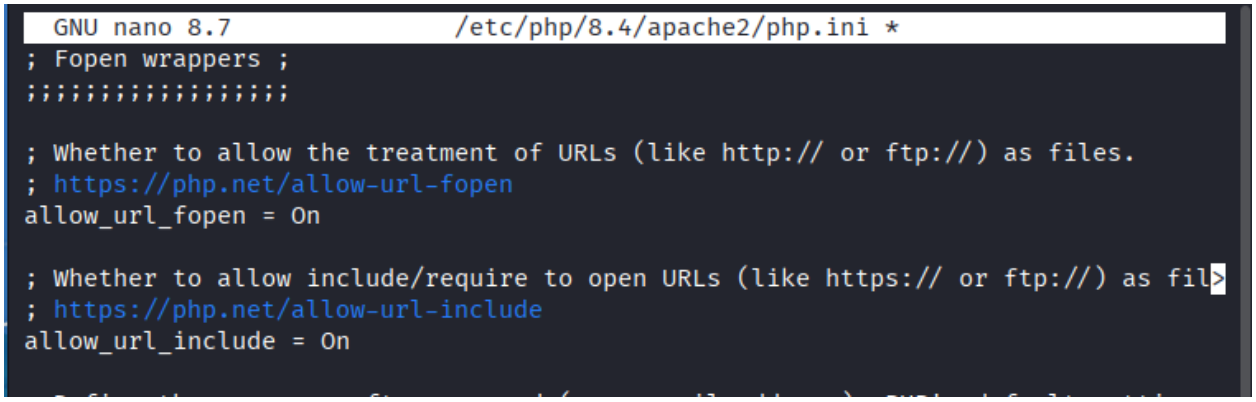
Navigate to the file after confirmation

sudo nano /etc/php/8.4/apache2/php.ini

Adjusting some configure line for file inclusion:

allow_url_fopen = On

allow_url_include = On



maximum_execution_time = 30

```
; Maximum execution time of each script, in seconds
; https://php.net/max-execution-time
; Note: This directive is hardcoded to 0 for the CLI SAPI
max_execution_time = 30
```

Memory_limit = 128M

```
; Maximum amount of memory a script may consume
; https://php.net/memory-limit
memory_limit = 128M
```

Restart Apache to apply changes:

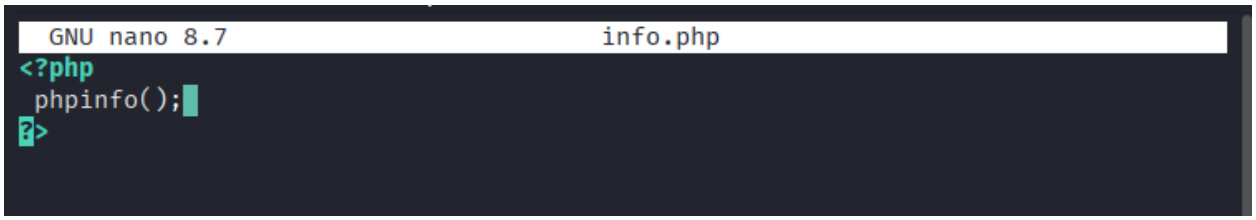
sudo systemctl restart apache2

To verify that php is correctly configured and loaded by apache, created a php file and paste the following command

<?php

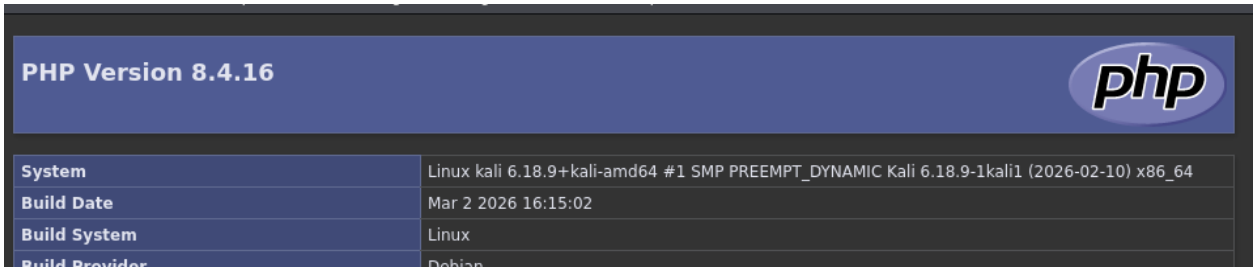
phpinfo();

?>



Checking in the browser

http:127.0.0.1/info.php



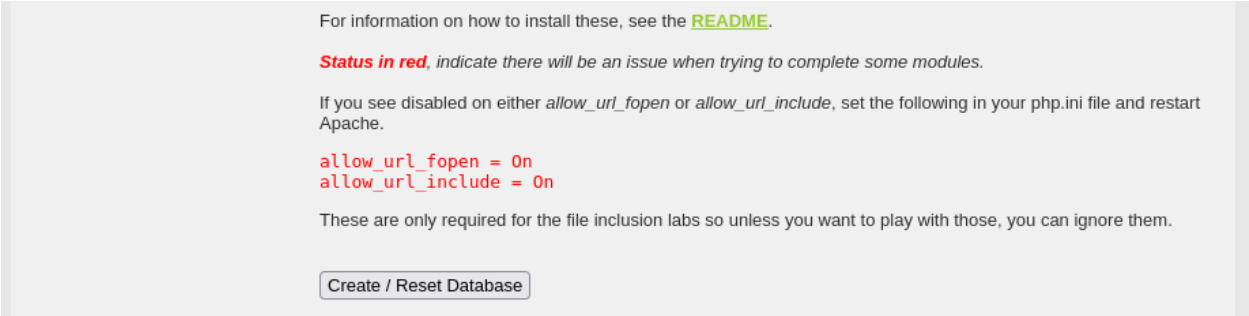
Now it confirmed that php and apache are configured and loaded correctly

Step 5: Final Web Setup

Assessing the DVWA interface by exploiting the CSRF, Open your browser and go to:

http://localhost/dvwa/setup.php

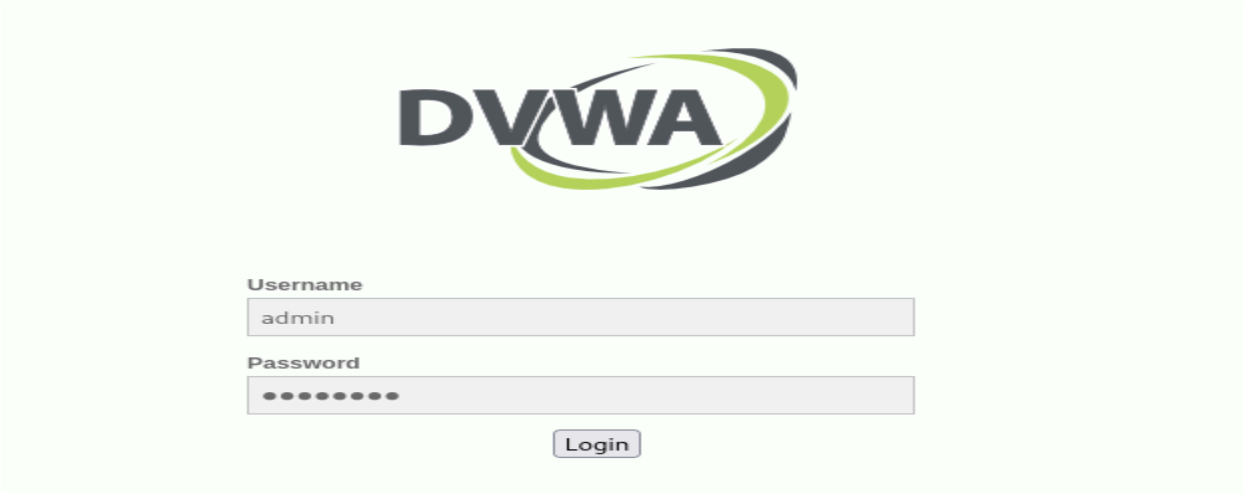
Create / Reset Database" button.



Logging with the defaults:

Default Username: admin

Default Password: password



Now we have access to the DVWA interface

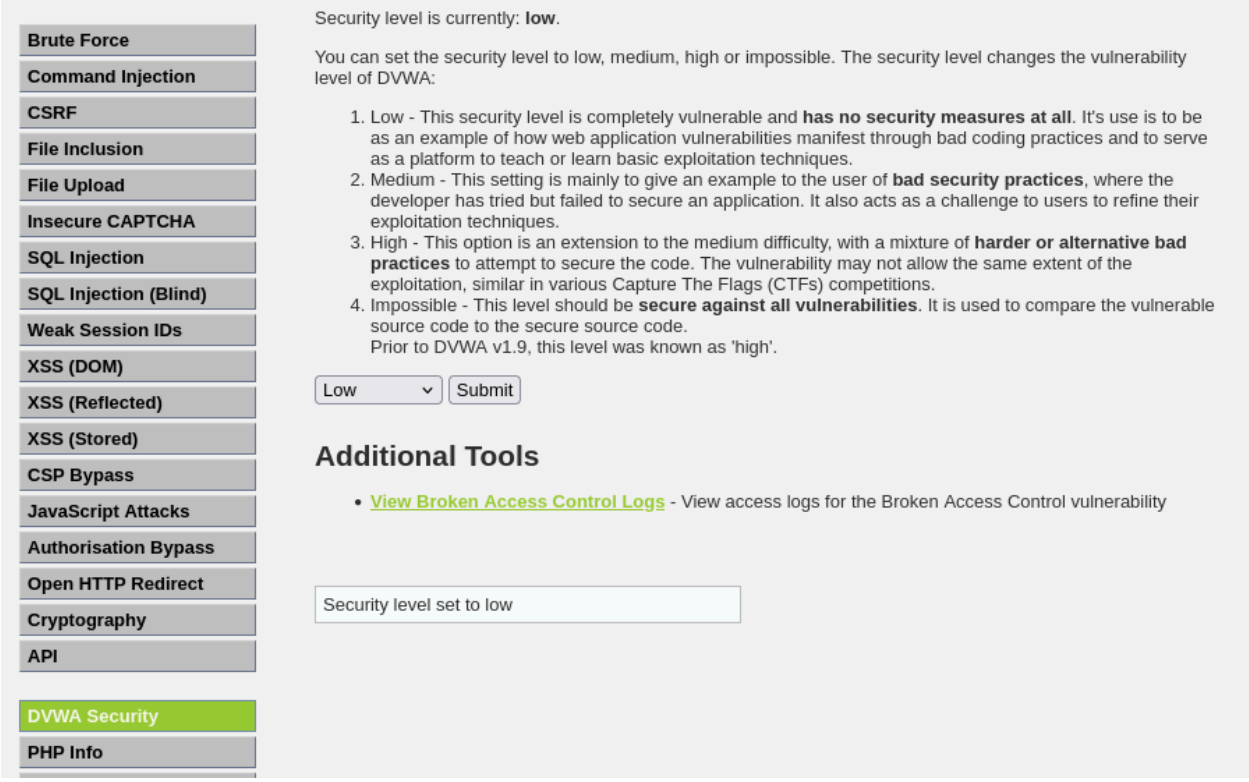


Step 6: Adjusting Security Levels

Dvwa has security level which demonstrate how severe an attack can be

Once logged in, look at the left-hand sidebar for **"DVWA Security"**.

- **Low:** No security. Good for understanding the raw vulnerability.
- **Medium:** Basic filters (e.g., blacklisting "script" tags). Good for learning bypasses.
- **High:** Advanced filters and logic.
- **Impossible:** Secure code. Use this to see how the code *should* be written to prevent attacks.



Step 7: Testing scenario

Sql injection： Testing a login or user ID field in DVWA.

- **Security Level** → set to **Low**
- Go to: **Vulnerabilities** → **SQL Injection**

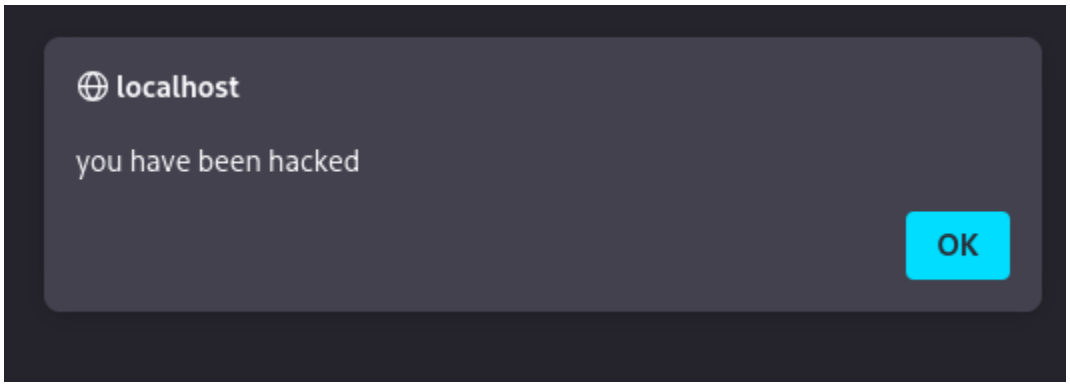
- **Test Input Field:** entering 1' OR '1'='1

Output below: this executed due to lack of **input validation and sanitized statement**



Xxs: Testing input fields that reflect user input (name or comment field)

inject Script: <script>alert('you have been hacked')</script>



To prevent the above scenario ensure to implement:

- Input validation
- Sanitized statement
- Html encoding
- Content security policy (CRP)

⚠️ **Critical Warning:** **Never** install DVWA on a server that is accessible from the internet. It is designed to be hacked; if you put it on a public server, someone **will** take over your machine within minutes. Always keep it in a local VM or an isolated environment.